PROJECT REPORT: AIR QUALITY PREDICTOR

INTRODUCTION

The **Air Quality Predictor** is a Python-based computational tool designed to estimate and categorize atmospheric conditions. As air pollution becomes a global health crisis, there is a growing need for accessible tools that can interpret complex sensor data. This project provides a simplified interface where users can input environmental variables like Temperature, Humidity, and PM2.5 levels to receive an immediate assessment of the air quality index (AQI) and associated health risks.

PROBLEM STATEMENT

In many urban areas, raw data from environmental sensors (like PM2.5 concentrations) is available, but it is often difficult for the general public to understand what these numbers mean for their health. There is a lack of simple, localized applications that can:

- Convert raw pollutant data into a standardized AQI.
- Provide immediate health advisories based on specific data points.
- Operate without complex hardware or expensive proprietary software.

OBJECTIVES

- To develop a functional Python script that calculates AQI based on environmental inputs.
- To implement conditional logic that categorizes air quality into "Good," "Moderate," "Poor," or "Hazardous."
- To provide automated health recommendations based on the calculated results.
- To create a user-friendly Command Line Interface (CLI) for data entry.

WHY THIS PROJECT MATTERS

Air pollution is often an "invisible killer." This project matters because it democratizes data. By creating a tool that translates PM {2.5} the most dangerous fine particulate matter into a readable scale, we empower individuals to make decisions, such as whether to exercise outdoors or use air purifiers, potentially reducing respiratory and cardiovascular health issues.

SCOPE OF THE PROJECT

The current scope of this project includes:

- **Manual Data Entry:** Processing user-provided values for temperature and pollutants.
- **Mathematical Modelling:** Using a linear formula to estimate the AQI.

- **Health Logic:** Mapping numerical outputs to the standard AQI color-coded categories.
- **Localized Use:** Designed for individual or classroom-level environmental monitoring.

METHODOLOGY

The project follows a **Structured Programming Approach**:

1. **Data Acquisition:** Using the input() function to gather environmental parameters.
2. **Normalization:** Converting inputs into floating-point numbers for calculation.
3. **Algorithmic Processing:** Applying the formula AQI = (PM {2.5} \times 1.5) + (Temp \times 0.1) to generate a weighted result.
4. **Classification:** Using a multi-way selection structure (if-elif-else) to determine the safety category.

IMPLEMENTATION

The project was implemented using Python 3.12 in the VS Code environment. The code utilizes basic data types (floats, strings) and control flow structures. No external heavy libraries were used to keep the program lightweight and ensure it runs on any system with a standard Python installation.

AI/ML RELEVANCE

While this specific version uses a Rule-Based System, it serves as the essential first step toward Machine Learning. In ML, we would replace the hardcoded formula with a Linear Regression model that "learns" the relationship between Temperature, Humidity, and AQI from a historical dataset. This project establishes the "Features" (Inputs) and "Labels" (Outputs) required for future AI implementation.

CHALLENGES FACED

- **Formula Accuracy:** Determining a simple mathematical formula that reasonably reflects real-world AQI trends.
- **Input Validation:** Handling cases where a user might enter non-numeric data, which would cause the program to crash.
- **Logic Thresholds:** Aligning the if-else categories with official environmental protection standards.

RESULTS

The program successfully takes real-time inputs and produces an AQI value.

- **Input:** Temp: 30, Humidity: 50, PM2.5: 120.
- **Output:** Calculated AQI: 183.0 (Category: Polluted/Mask Required).

The system demonstrates 100% consistency in mapping values to their correct safety categories.

LIMITATIONS

- **Static Calculation:** The formula is a simplified model and doesn't account for wind speed or NO2/SO2 levels.
- **Lack of Persistence:** The data is not saved to a database for historical trend analysis.
- **Manual Input:** It requires the user to know the PM2.5 levels from an external source.

FUTURE ENHANCEMENTS

- **IoT Integration:** Connecting the code to a physical sensor (like an MQ-135) for live data.
- **Database Support:** Using SQL to store daily air quality logs.
- **Predictive Analysis:** Using a Linear Regression model to predict tomorrow's air quality based on today's weather patterns.

WHAT I LEARNED

Through this project, I strengthened my understanding of:

- Python syntax and conditional logic.
- The importance of data weighting in mathematical formulas.
- The "Input-Process-Output" (IPO) cycle in software development.
- How to document technical code for a project report.

CONCLUSION

The Air Quality Predictor is a successful demonstration of how basic programming can solve environmental awareness problems. It provides a reliable framework for categorizing air health and serves as a vital stepping stone toward more complex Artificial Intelligence and IoT-based environmental solutions.